

Worksheet No. – 06

Student Name: Sukhleen Kaur

UID: 25MCA20031

Branch: MCA

Section/Group: 25MCA-6B

Semester: 2nd

Date of Performance: 09.04.2026

Subject Name: Design and Analysis of Algorithm Lab **Subject Code:** 25CAP-612

Aim/Overview of the practical: Find a subset of a given set $S = \{s_1, s_2, \dots, s_n\}$ of n positive integers whose sum is equal to a given positive integer d . For example, if $S = \{1, 2, 5, 6, 8\}$ and $d = 9$ there are two solutions $\{1, 2, 6\}$ and $\{1, 8\}$. A suitable message is to be displayed if the given problem instance doesn't have a solution.

Objective: To design and implement an algorithm that identifies all possible subsets of a given set of positive integers whose sum equals a specified target value. The program should efficiently explore combinations, display all valid subsets that satisfy the condition, and provide an appropriate message when no such subset exists.

Procedure/Algorithm/Code

Algorithm:

1. Read n , set S , and target sum d
2. Initialize $sum = 0$, $index = 0$, $found = false$
3. Use recursive function $subsetSum(index, sum)$
4. If $sum == d \rightarrow$ print subset, set $found = true$
5. If $index == n$ or $sum > d \rightarrow$ return
6. Include element $\rightarrow$ call $subsetSum(index+1, sum + S[index])$
7. Exclude element $\rightarrow$ call $subsetSum(index+1, sum)$
8. Repeat for all elements
9. If no subset found $\rightarrow$ print "No solution"

Code:

```
#include <stdio.h>
```

```
int main() {
```

```
int n, d;

printf("Enter number of elements: ");

scanf("%d", &n);

int S[n];

printf("Enter elements: ");

for(int i = 0; i < n; i++) {

    scanf("%d", &S[i]);

}

printf("Enter target sum: ");

scanf("%d", &d);

// DP table

int dp[n+1][d+1];

// Initialize

for(int i = 0; i <= n; i++) {

    for(int j = 0; j <= d; j++) {

        if(j == 0)

            dp[i][j] = 1; // sum 0 is always possible

        else if(i == 0)

            dp[i][j] = 0; // no elements → no sum

    }

}

// Fill DP table

for(int i = 1; i <= n; i++) {

    for(int j = 1; j <= d; j++) {

        if(S[i-1] <= j)
```

```
dp[i][j] = dp[i-1][j] || dp[i-1][j - S[i-1]];
```

```
else
```

```
dp[i][j] = dp[i-1][j];
```

```
}
```

```
}
```

```
// Check result
```

```
if(dp[n][d]) {
```

```
    printf("Subset with given sum exists.\n");
```

```
    // Backtracking to print subset
```

```
    int i = n, j = d;
```

```
    printf("Subset: ");
```

```
    while(i > 0 && j > 0) {
```

```
        if(dp[i][j] != dp[i-1][j]) {
```

```
            printf("%d ", S[i-1]);
```

```
            j -= S[i-1];
```

```
        }
```

```
        i--;
```

```
    }
```

```
} else {
```

```
    printf("No subset with given sum exists.\n");
```

```
}
```

```
return 0;
```

```
}
```

Output

```
Enter number of elements: 5
Enter elements: 6 8 9 4 7
Enter target sum: 11
Subset with given sum exists.
Subset: 7 4
```

Learning outcomes (What I have learnt):

1. Understood the concept of the Subset Sum Problem
2. Learnt how to use backtracking to explore all possible subsets
3. Developed skills in recursive problem solving